GRAFOS

Canon: ids de árvores enraizadas

Como a lista ordenada dos filhos identifica uma subárvore

Ester dos Santos da Silva · 2324328
Leonardo Enzo Pinheiro Diogenes · 2420372

CONTEXTO E SAÍDA DO CANON

Duas árvores entram, o canon devolve um id por raiz

O problema

- Duas árvores com n nós, raiz no vértice 1
- Saída: SIM se forem isomorfas, NÃO se não forem
- canon(g, mapa) devolve o id da raiz

Entrada de exemplo (n = 6)

- A: 1 → {2 → {4, 5}, 3 → {6}}
- B: 1 → {3 → {4, 5}, 2 → {6}}
- Mesma forma, rótulos trocados

Saída do programa (versão com depuração)

```
ids de A (posição i = vértice i+1): [3, 2, 1, 0, 0, 0]
ids de B (posição i = vértice i+1): [3, 1, 2, 0, 0, 0]
mapa (por id): {[]=0, [0]=1, [0, 0]=2, [1, 2]=3}
ra = 3, rb = 3  →  SIM
```

Saída de uma versão de depuração do código, que imprime o vetor id[] dentro de canon e o mapa no main.

Ester dos Santos da Silva · Leonardo Enzo Pinheiro Diogenes

COMO O ID É ENCONTRADO

O id de um vértice vem da lista ordenada dos ids dos filhos

Vértice (árvore A)	Filhos	Lista ordenada	Id
4, 5, 6	nenhum	[]	**0, lista nova**
3	6	[0]	**1, lista nova**
2	4, 5	[0, 0]	**2, lista nova**
1	2, 3	[1, 2]	**3, lista nova**

Lista já vista: reutiliza o id. **Lista nova:** recebe mapa.size().
Ordenar: [2, 1] e [1, 2] são o mesmo multiconjunto. **Mapa compartilhado:** os ids de A e de B são comparáveis.

Na árvore B, os vértices 2 e 3 trocam de id, mas a raiz tem a mesma lista [1, 2] e recebe o id 3.

Ester dos Santos da Silva · Leonardo Enzo Pinheiro Diogenes

CÓDIGO DO CANON · 1 DE 2

Primeiro, a ordem em que os vértices serão processados

Main.java · canon() 1/2

```java
private static int canon(Graph g, Map<List<Integer>, Integer> mapa) {
    int n = g.V();
    int[] pai = new int[n];
    int[] ordem = new int[n];
    int k = 0;
    pai[0] = -1;
    Stack<Integer> pilha = new Stack<Integer>();
    pilha.push(0);
    while (!pilha.isEmpty()) {
        int v = pilha.pop();
        ordem[k++] = v;
        for (int w : g.adj(v)) {
            if (w != pai[v]) {
                pai[w] = v;
                pilha.push(w);
            }
        }
    }
```

ordem[] guarda a pré-ordem: todo filho aparece depois do pai. w != pai[v] evita voltar ao pai, e é isso que dispensa o marked[].

CÓDIGO DO CANON · 2 DE 2

Depois, o id de cada vértice, dos filhos para o pai

Main.java · canon() 2/2

```
1   int[] id = new int[n];
2   for (int i = n - 1; i >= 0; i--) {
3       int v = ordem[i];
4       int numFilhos = g.degree(v) - (v == 0 ? 0 : 1);
5       Integer[] chave = new Integer[numFilhos];
6       int j = 0;
7       for (int w : g.adj(v)) {
8           if (w != pai[v]) chave[j++] = id[w];
9       }
10      Arrays.sort(chave);
11      List<Integer> lista = Arrays.asList(chave);
12      Integer val = mapa.get(lista);
13      if (val == null) {
14          val = mapa.size();
15          mapa.put(lista, val);
16      }
17      id[v] = val;
18  }
19  return id[0];
```

Passo a passo

- ordem[i] de trás para frente: filhos antes do pai
- chave: ids dos filhos, já calculados
- sort: escolhe um representante do multiconjunto
- get / put: id existente, ou mapa.size() se for novo
- id[0]: id da raiz

JUSTIFICATIVA DA COMPLEXIDADE

Um único fator log: a ordenação dos filhos

Trecho de canon	Tempo	Por quê
DFS (laço while)	**O(n)**	cada vértice entra e sai da pilha uma vez; soma dos graus = 2(n−1)
Montar chave	**O(n)**	percorre adj(v) de cada vértice: soma dos graus = 2(n−1)
Arrays.sort	**O(n log n)**	soma de c·log c, com c filhos por vértice, soma de c = n−1
mapa.get e put	**O(n) esperado**	custo = tamanho da chave; soma das chaves = n−1
Total	**O(n log n)**	memória O(n): pai, ordem, id, pilha e chaves do mapa

main chama canon duas vezes, então o custo dobra, sem mudar a ordem. O O(n) esperado do mapa supõe bom espalhamento do hash.